AI FOR YOUR WORK

How it really works, how to make it work for you

TWO KINDS OF AI WORKFLOWS:

AI FOR YOUR WORK (TODAY)

AI IN YOUR PRODUCT (TUESDAY)

HOW CLAUDE CODE ACTUALLY WORKS

THE AGENTIC LOOP

Claude Code is a **conversation loop** that wraps the Claude API.

You type → HARNESS → API call → Claude responds → HARNESS → Tool calls → API call → Final answer

- **Every turn is a fresh, stateless API call.** The model has no memory. The harness replays the full history on every single message.
- **Tools are JSON schemas.** Claude outputs a request: `read_file("app.py")`. The harness runs it. Result goes back as a new message.
- **The loop continues** until Claude has nothing left to do. A complex task might take 10-20+ loops.
- **The model is the engine. The harness is the car.** You don't drive an engine.

💡 What users call "Claude Code being great at coding" is actually the **harness**: file access, tool permissions, context management. The model is identical to the one in claude.ai. The wrapper is the product.

THE MODEL VS. THE HARNESS

🧠

THE MODEL (LLM)

- **Stateless.** No memory between turns.
- Can't read files, run code, or browse the web
- Just predicts the next token
- **Identical** in claude.ai, Claude Code, Cursor, Copilot

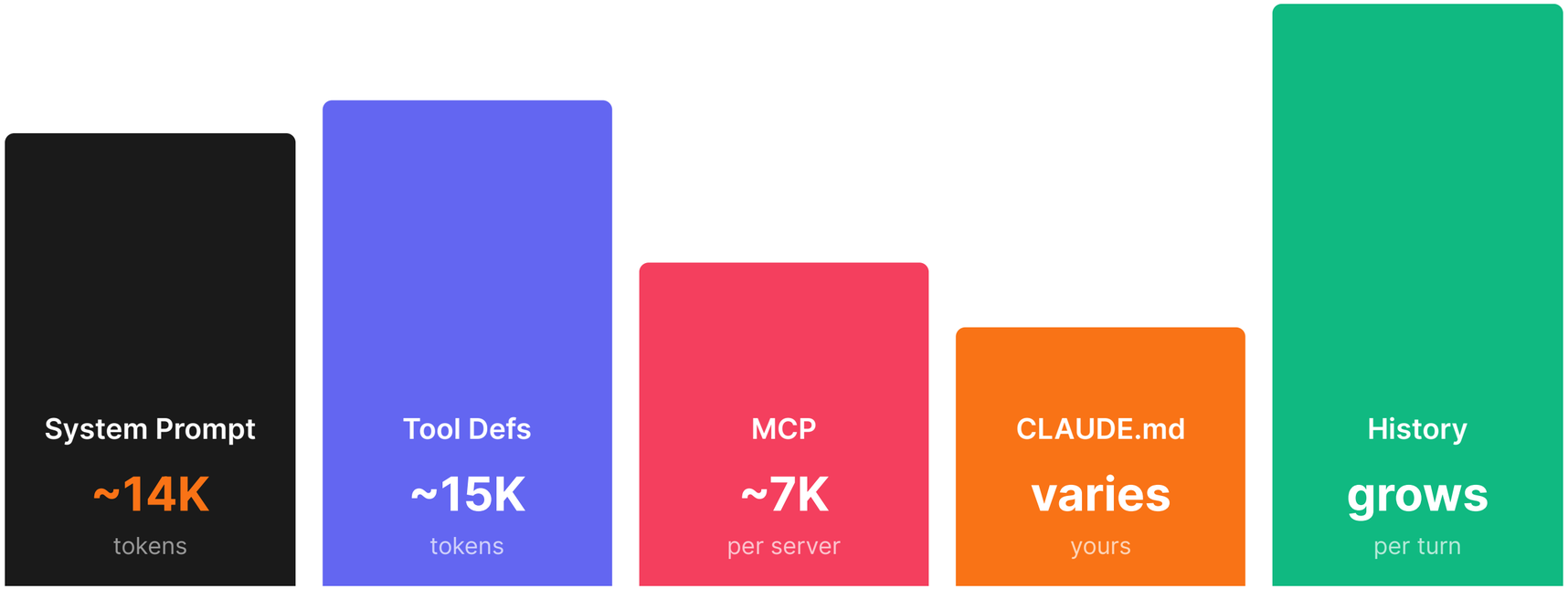
🚗

THE HARNESS (CLAUDE CODE)

- Stores and **replays** conversation history
- Executes tools (Read, Edit, Bash, MCP)
- Manages permissions and safety
- **This is the product.** This is what differs.

💡 **The model is the engine. The harness is the car.** Nobody drives an engine. Claude Code, Cursor, and Copilot all use the same engines — they compete on the harness.

WHAT'S IN EVERY API CALL



Before you type a single word, **~36K+ tokens** are already spoken for. Every turn. And that full payload is **re-sent every message.**

⚠ **MCP bloat is real.** Developers have reported 66,000–144,000 tokens consumed by MCP tools alone. Claude Code mitigates this with "deferred tools" — only loading 3-5 relevant schemas per query (~85% reduction).

THE CONTEXT WINDOW ILLUSION

WHAT YOU THINK

200,000
tokens of space for your work

WHAT YOU GET

~150,000
usable tokens before compaction

- **Compaction triggers at ~95% capacity.** Claude Code compresses your conversation. **Lossy.**
- More overhead = less room before compaction = lost context sooner
- A bloated CLAUDE.md doesn't just cost money. It **steals working memory.**
- **Context rot:** accuracy and recall degrade as context grows — before you even hit the limit.
- **Lost in the middle:** LLMs attend most to beginning and end. Critical instructions buried mid-context get ignored.

THE CONFIGURATION LAYERS

Tuning the Engine

FOUR MECHANISMS

	CLAUDE.MD	SKILLS	AGENTS	HOOKS
What	Always-on context	On-demand workflows	Isolated workers	Shell scripts
Loaded	Every API call	When invoked	When spawned	Every trigger
Context	Shared (main)	Shared (main)	**Fresh window**	N/A (shell)
Compliance	High (primacy)	Medium (middle)	Highest (fresh)	100% (deterministic)
Use for	Identity, rules	Workflows, reference	Parallel tasks	Non-negotiable rules

💡 **Lost in the Middle:** Skills load mid-conversation. LLMs pay less attention to the middle of context. That's why skill instructions get followed less reliably than CLAUDE.md.

WHAT GOES WHERE

DOES IT APPLY TO EVERY CONVERSATION?

Yes → CLAUDE.MD **No** → keep going

IS IT A MULTI-STEP WORKFLOW WITH HUMAN GATES?

Yes → SKILL **No** → keep going

CAN IT RUN AUTONOMOUSLY IN ISOLATION?

Yes → AGENT **No** → probably a skill

MUST IT EXECUTE 100% OF THE TIME, NO EXCEPTIONS?

Yes → HOOK (deterministic, not probabilistic)

WHAT THE PROS PUT IN CLAUDE.MD

AUTHOR	LINES	WHAT'S IN IT
Simon Willison	30	Build commands + minimal instructions
Anthropic (their own repo)	51	Commands, architecture, "Things That Will Bite You"
Boris Cherny	59	Behavioral rules distilled into principles
Dan Abramov (React core team)	74	Architecture overview, dev commands, conventions
Andrej Karpathy	72	Think before coding, simplicity, surgical changes

The pattern: 30-75 lines. Project identity + dev commands + behavioral rules. Workflows go in skills.

THINGS THAT WILL BITE YOU

LLMS CANNOT DO MATH

They don't calculate. They predict the next likely token.

WHAT HAPPENS

"What's 17% of 4,823?"
The LLM will **confidently give you a wrong number.** It looks right. It's not.

THE FIX

calc-mcp (MCP server)
Every calculation routes to `mcp__calc-mcp__math`. Math, dates, unit conversions.

From Ken's CLAUDE.md:

```
"ALL math and date calculations MUST use calc-mcp.
No exceptions. Never do mental math. Never estimate."
```

Mechanism: CLAUDE.MD rule + MCP server (always available, every conversation)

LLMS DON'T KNOW WHAT DAY IT IS

The model has a training cutoff. It doesn't have a clock.

WHAT HAPPENS

"Schedule this for next Tuesday."
The LLM doesn't know today's date. It will guess or hallucinate.

THE FIX

A hook. Runs a Python script on every message.
Output gets injected before Claude sees your prompt.

settings.json (the hook):

```
"UserPromptSubmit": [{
  "hooks": [{
    "type": "command",
    "command": "python3 dateutil.py --now"
  }]
}]
```

→ "Thursday March 26, 2026 - 08:30 AM"

Mechanism: HOOK (deterministic). Not "Claude, please check the date" (works 90%). The hook fires **100%.**

LLMS CAN'T TOUCH THE REAL WORLD

Without tools, an LLM is a brain in a jar.

BROWSER TESTING

Playwright skill

Writes and runs browser automation. Tests pages, fills forms, takes screenshots.

BROWSER CONTROL

Chrome MCP

Navigate real tabs. Read pages, click, fill forms. Live interaction.

EXTERNAL SERVICES

MCP servers

Gmail, Google Calendar, Todoist. Read and act on your behalf.

Mechanism: TOOLS + MCP. Each is a JSON schema in the system prompt. Claude requests it; the harness executes it.

LIVE WALKTHROUGH

Ken's Real Setup

SKILL: /RESEARCH-REPORT ~250 LINES

1. **BRIEF CREATION**
   Define research questions, scope, methodology
2. **HUMAN REVIEW** STOP
   Ken approves the brief before research begins
3. **RESEARCH EXECUTION**
   WebSearch, WebFetch, spawns **research subagents** for parallel deep dives
4. **SOURCE VERIFICATION**
   Chains to `/verify-sources` → spawns **source-checker agent** per citation
5. **HUMAN REVIEW** STOP
   Ken reviews report, provides edits. All edits re-verified against sources.
6. **AUDIO + TASKS**
   Generate listening script, auto-create Todoist task

The skill is my workflow as code. Without it, I'd forget steps, skip verification, be inconsistent.

SKILL: /FEATURE-FORGE ~209 LINES

1. Brief → 2. Architecture → 3. Plan → Doc Review 🤖 → 4. Implement → Code Review 🤖 → 5. Push → 6. Cleanup

HUMAN GATES BETWEEN EVERY PHASE

`STOP — Brief phase complete. Ready for human review.` appears after each phase. Claude cannot proceed until the human approves.

SUBAGENTS DO THE REVIEWS

🤖 = `code-reviewer` agent. Spawned automatically after Plan and after Implement. Separate context, focused criteria.

Same pattern, different domain. Multi-step workflow. Human gates for judgment. Agents for delegation.

TWO AGENT PATTERNS

MINIMAL WORKER: SYNTHETIC-EVAL

17 lines · model: Sonnet · tools: Read

```
1. Read the prompt file you are given
2. Follow the instructions exactly
3. Return ONLY the JSON object

Stay in character. Lean negative
to counter AI sycophancy bias.
```

Spawned dozens of times in parallel. Each one embodies a different persona.

SPECIALIST: SOURCE-CHECKER

137 lines · model: Sonnet · tools: Bash, Read, Write, Grep, Chrome

```
Three-tier fetch strategy:
1. curl (fast, simple)
2. Playwright (JS-rendered pages)
3. Chrome MCP (last resort)

Returns: VERIFIED | INACCURATE
         NOT_FOUND | BROKEN_LINK
```

Spawned per citation by /verify-sources. Has specific tool access.

CLAUDE.MD DEFINES.

SKILLS ORCHESTRATE.

AGENTS EXECUTE.

HOOKS GUARANTEE.

WHAT TO BUILD NEXT

Feature Prioritization

FRAMEWORK 1: IMPACT/EFFORT MATRIX

QUICK WINS

High impact, low effort

Do these first

BIG BETS

High impact, high effort

Plan carefully

FILL-INS

Low impact, low effort

If you have time

MONEY PITS

Low impact, high effort

Don't

💡 **"Impact" is whatever matters most to your goals.** Revenue? User retention? Learning speed? Define it first, then ladder every feature's impact back up to those goals.

Best for: Gut-level sorting when overwhelmed with options. Fast, visual, gets the team aligned.

FRAMEWORK 2: RICE SCORING

Reach × Impact × Confidence ÷ Effort = Score

REACH

How many users affected?
(e.g., 100 users/week)

IMPACT

How much does it move the needle?
0.25 (minimal) → 3 (massive)

CONFIDENCE

How sure are you?
100% (data) → 50% (gut feel)

EFFORT

Person-hours to build.
Be honest about your schedule.

Best for: When gut feel disagrees within the team. Forces assumptions to be explicit.

EXERCISE: PRIORITIZE YOUR BACKLOG

INPUTS

- Peer review feedback (Tuesday)
- Your roadmap homework
- AI feature ideas from today
- Your own backlog

OUTPUTS

- Top 5-7 items ranked
- Your **#1 priority** starred
- Bonus: tag items where AI could help

Pick **one framework**. You have **10 minutes.**

💡 Knowing what you're **not** building is as important as knowing what you are.

EXAMPLE: A REAL STARTUP BACKLOG

Text Affirmations — an AI-powered SMS product. Here's what the actual backlog looks like:

TASK	CATEGORY	PRIORITY
Fix payment abandonment flow: add "Resume Payment" path	BUG	🔴
LLM can take action to set delivery windows	FEATURE	🔴
Set up eval pipeline for text quality improvement	FEATURE	🔴
Figure out urgency angle for signups	GROWTH	🔴
Pick next track, build landing page + ad campaign	GROWTH	🔴
Put affirmations as interstitials in onboarding quiz	FEATURE	🔴
Fix referral clawback service (11 failing specs)	BUG	🟡
Upgrade Ruby 3.2 → 3.3+ (15-20% perf gain)	INFRA	🟡
Record first batch of Instagram reels (8 scripts ready)	GROWTH	⚪
Say "you mentioned" instead of "you are" in LLM responses	QUICK WIN	⚪
Add parallax scrolling to homepage images	QUICK WIN	⚪

39 tasks total. This is 11. You can't do them all. *What do you cut?*

WHAT'S NEXT

FOR TUESDAY

DUE MAR 31

PRIORITIZED ROADMAP

- Take your #1 priority from today's exercise and turn it into a **build plan**
- What are you building first? What are you cutting? What's deferred?
- Commit the roadmap to your repo

RECOMMENDED

Claire Vo's *How I AI* podcast. Live demos of real AI workflows. YouTube/Spotify.

END OF CLASS 17

Next: AI in Your Product (Tuesday, Mar 31)